✈️ TECHNICAL HIRING CHALLENGE

SkyRoute Travel Platform

Full-Stack Engineering Assessment | Senior Level

⏱️ Estimated Time	🎯 Level	🤖 AI Tools
3 – 4 hours	Senior Full-Stack	Allowed & encouraged

1. Overview & Context

You have been brought in as a senior engineer to build a core feature for SkyRoute — a travel aggregator platform that allows users to search, compare, and book flights. The business is growing fast and needs a well-architected, maintainable system that can evolve over time.

Your task is to build a Flight Search & Booking module — a small but real-world slice of the platform, including a frontend UI and a backend API. We expect production-quality thinking in the code and architecture you produce.

> 💡 **Note on AI Tools**
> AI coding tools (Copilot, Cursor, ChatGPT, Claude, etc.) are fully allowed and encouraged. You will be asked to walk through your codebase and explain it during the interview

2. Business Context

SkyRoute aggregates flight data from multiple airline providers. Each provider exposes its own API with its own pricing model. For this challenge, you will integrate with two providers: GlobalAir and BudgetWings. Since their real APIs are not accessible, you must mock them in your backend — each mock should return a realistic set of flight results for any given search.

Each provider applies its own rules to calculate the final price shown to users:

Provider	Pricing Rule
GlobalAir	Base fare + 15% fuel surcharge. Always round the final price to 2 decimal places.
BudgetWings	Base fare − 10% promotional discount. The minimum final price is $29.99. The discount is always applied to the base fare only

> ⚠ **Important**
> The platform expects to onboard additional airline providers in the future.

3. Functional Requirements

3.1 Flight Search

Build a flight search form that lets users find available flights. The form must capture:

- Origin and destination airports (a simple dropdown is fine — no live airport API needed; hardcode at least 6 airports across at least 2 countries)
- Departure date
- Number of passengers (1 to 9)
- Cabin class: Economy, Business, or First Class

On submission, the frontend calls the backend and displays the results as a list or table showing: airline provider, flight number, departure time, arrival time, duration, cabin class, and price.

> ⚠ **Pricing Display**
> The price shown in results must be the TOTAL price for all passengers combined. The per-passenger price should also be visible as secondary information (e.g., 'USD 320.00 total / USD 160.00 per person'). These are two different numbers — make sure the UI makes that distinction clear.

3.2 Flight Results & Sorting

- Results must be sortable by: Price (low to high and high to low), Duration (shortest first), and Departure time
- Sorting must happen on the frontend — no additional API call should be made when changing sort order
- Show a loading indicator while the search is in progress
- Show a clear empty state if no flights match the search

3.3 Booking Flow

When a user selects a flight, display a booking screen that includes:

- A summary of the selected flight (route, provider, times, cabin class)
- A price breakdown showing per-passenger price, number of passengers, and total
- A passenger details form with: full name, email address, and a document number field
- A Confirm Booking action that submits to the backend and returns a booking reference code

> ⚠ **Document Type Requirement**
> For international flights (origin and destination in different countries), the document number field must be labelled 'Passport Number' and validated accordingly. For domestic flights, it must be labelled 'National ID' instead. Both the label and the validation rule must change based on the selected route.

3.4 Backend API

Design and implement a backend API that supports the flight search and booking flows described above, design the structure, endpoints, and contracts.

4. Technology

> 🔧 **Stack Choice**
>
> Angular + .NET

5. Deliverables

1. A working application (frontend + backend) that runs locally
2. Source code in a public or shared Git repository
3. A README with: setup and run instructions, a brief description of the architecture decisions you made, and any trade-offs or known limitations

> 📌 **Scope Notes**
> You do not need to deploy to any cloud environment. Running locally is sufficient.
> If you run out of time on any part, document what you would have done in the README.

6. What to Expect at the Interview

The review session will last approximately 45–60 minutes. You will walk us through your codebase live.

Good luck.

Questions about the challenge? Reach out to your recruiter.